evaluate.py — technical

scripts/evaluate.py · 275 lines · scripts/evaluate.py

Purpose

Stage evaluation and the comparison report.

Step by step

Classes and functions in source order. Signatures, line ranges, raised exceptions and settings are extracted from the code; numbered steps describe what the code does.

`_model_dir`

```
def _model_dir(paths, stage: str)
```

lines 62-70 · raises SystemExit · settings REPLACEMENT_METHOD

1. `after_training` is only valid for the bias method; returns `replaced-model/` or `trained-model/` (must exist).

`run_stage`

```
def run_stage(stage: str, cfg, paths, examples, settings, tokenizer, task) -> dict
```

lines 73-137 · raises SystemExit · settings EVAL_BATCH_SIZE, EVAL_QUALITATIVE_SAMPLES, MODEL_PATH or HF_MODEL_ID, REPLACEMENT_METHOD

Load the stage's model, evaluate it and write its metrics.

1. `original`: load the original model. Otherwise load the stage checkpoint with the configured placement; its replacement method and model source must match the configuration; open the original checkpoint and build the reference probe for the removed experts.
2. `evaluate_model` (with the probe for pruned stages), then `task.qualitative` on the first `EVAL_QUALITATIVE_SAMPLES` examples.
3. Write `<stage>_metrics.json` (task, metric definition, losses, perplexity, probe results, settings, model info, replaced experts, method, runtime, memory, environment, qualitative) and `<stage>_per_example.pt`.
4. Delete the model and empty caches before returning (only one model in memory at a time).

`_baseline_valid`

```
def _baseline_valid(paths, settings) -> bool
```

lines 140-142

1. The original metrics exist and were computed with the same evaluation fingerprint.

`write_comparison`

```
def write_comparison(paths, settings) -> None
```

lines 145-248 · raises RuntimeError

Combine all evaluated stages into comparison.json / comparison.md.

1. Load every stage's metrics with the current fingerprint (others are ignored with a warning).
2. Per stage: losses, change vs original; for pruned stages, after checking identical example ids: per-example deltas and top-1 agreement with the original predictions.
3. Merge qualitative rows by example; write `comparison.json` and a Markdown report (settings, metric definition, loss table with perplexity column for causal models, per-layer reconstruction table, qualitative examples).

`main`

```
def main() -> None
```

lines 251-271

Script entry point: runs this stage end to end.

1. Load tokenizer and task; build/load the cached evaluation examples.
2. `--stage original`: evaluate unless a valid baseline exists (or `--force-baseline`).

3. Other stages: evaluate the original first if needed, then the requested stage; finally write the comparison.

Module constants

Name	Value
`STAGES`	('original', 'before_training', 'after_training')
`STAGE_TITLES`	{'original': 'Original model', 'before_training': 'Pruned (before training)', 'after_training': 'Pruned (after...

Errors raised

Function	Exception	Message (f-string fields in braces)
`_model_dir`	SystemExit	REPLACEMENT_METHOD={paths.cfg.replacement_method} has no training stage; use --stage before_training.
`run_stage`	SystemExit	Checkpoint uses REPLACEMENT_METHOD={manifest.get('replacement_method', 'bias')}, config has {cfg.replacement_method}. Re-run replace_experts.py.
`run_stage`	SystemExit	Checkpoint was built from {manifest['model_source']}, config uses {cfg.model_source}
`write_comparison`	RuntimeError	{stage} was evaluated on different examples than the baseline